

DSA Assignment Answers

SAQ'S :

1) Illustrate the following concepts with example

a) typedef :-

In C, typedef is used to give a new name to an existing data type, making it easier to read and understand.

Ex:-

```
#include <stdio.h>
```

```
typedef int Integer;
```

```
typedef struct {
```

```
    int x;
```

```
    int y;
```

```
} Point;
```

```
int main() {
```

```
    Integer a = 10;
```

```
    printf("a = %d\n", a);
```

```
    Point p1;
```

```
    p1.x = 5;
```

```
    p1.y = 10;
```

```
    printf("Point: (%d, %d)\n", p1.x, p1.y);
```

```
    return 0; }
```

b) enum :-

An enum (short for enumeration) is a user-defined data type in C that consists of a set of named integer constants. enum is often used to define a collection of related constants, improving readability and reducing the risk of errors.

Ex:-

```
#include <stdio.h>
```

```
enum Day {
```

```
    SUNDAY,
```

```
    MONDAY,
```

```
    TUESDAY,
```

```
    WEDNESDAY,
```

```
    THURSDAY,
```

```
    FRIDAY,
```

```
    SATURDAY
```

```
};
```

```
int main() {
```

```
    enum Day today;
```

```
    today = WEDNESDAY;
```

```
    printf("Today is: %d\n", today);
```

```
    if (today == WEDNESDAY) {
```

```

        printf("Midweek day.\n");
    }
    return 0;
}

```

2) Distinguish between structure and union.

Feature	Structure	Union
Memory Allocation	Allocates memory for all members individually.	Allocates memory equal to the size of the largest member.
Size	The size is the sum of all members' sizes.	The size is determined by the largest member.
Member Access	All members can be accessed independently at the same time.	Only one member can be accessed at a time.
Initialization	Multiple members can be initialized simultaneously.	Only the first member can be initialized.
Use Case	Used when all data members need to be stored simultaneously.	Used when memory needs to be saved and only one member is used at a time.

3) What is self-referential structure? Give an example.

Self Referential structures are those structures that have one or more pointers which point to the same type of structure, as their member.

Ex:-

```

struct Node {
    int data;
    Node* next;
};

```

4) What is a Data Structure. List out the types of Data Structures.

A **data structure** is a way of organizing and storing data to enable efficient access and modification.

Types of Data Structures:

1. Linear:

- Arrays
- Linked Lists
- Stacks
- Queues

2. Non-Linear:

- Trees
- Graphs

5) Compare Linear and Non Linear data structures

Feature	Linear Data Structures	Non-Linear Data Structures
Organization	Elements are arranged sequentially.	Elements are arranged in a hierarchical or interconnected manner.
Memory Allocation	Contiguous memory allocation.	Can use non-contiguous memory.
Traversal	Easier and straightforward (e.g., arrays, linked lists).	More complex (e.g., trees, graphs).
Examples	Arrays, Linked Lists, Stacks, Queues.	Trees, Graphs.
Access Time	Generally faster for direct access.	Access time can vary based on structure and traversal method.
Use Cases	Suitable for simple relationships and order.	Ideal for representing complex relationships and hierarchies.

6) List out differences between stack and queue.

Feature	Stack	Queue
Structure	Last In, First Out (LIFO)	First In, First Out (FIFO)
Access Method	Only top element is accessible.	Elements are accessed from both ends (front and back).
Main Operations	Push (add), Pop (remove).	Enqueue (add), Dequeue (remove).
Use Cases	Function calls, backtracking.	Scheduling, buffering.
Implementation	Can be implemented using arrays or linked lists.	Can also be implemented using arrays or linked lists.

7) Define abstract datatype : write ADT of stack and queue.

An **Abstract Data Type (ADT)** is a mathematical model for data types, where a data type is defined by its behavior (operations) rather than its implementation.

It specifies the operations that can be performed and the properties of the data, without detailing how those operations are implemented.

ADT of Stack

Operations:

- `push(item)`: Add an item to the top of the stack.
- `pop()`: Remove and return the item from the top of the stack.
- `peek()`: Return the item at the top of the stack without removing it.
- `isEmpty()`: Check if the stack is empty.
- `size()`: Return the number of items in the stack.

ADT of Queue

Operations:

- `enqueue(item)`: Add an item to the back of the queue.
- `dequeue()`: Remove and return the item from the front of the queue.
- `front()`: Return the item at the front of the queue without removing it.

- `isEmpty()`: Check if the queue is empty.
- `size()`: Return the number of items in the queue.

8) Write the prefix and postfix forms of the following expressions

a) $(A+B)*D+E/(F+A*D)+C$

Prefix : $+++ABD/E+F*ADC$

Postfix : $AB+D*EFAD*/+C+$

b) $A^B-C+D^*E-A^*C$

Prefix: $-+ - ^AB C ^*DE ^*AC$

Postfix: $AB^C-DE^*+AC^*-$

c) $(A + B) * C - (D - E) * (F + G)$

Prefix:

$- * + A B C * - D E + F G$

Postfix:

$A B + C * D E - F G + * -$

d) $(A+B)*D+E/(F+A*D)+C$

Prefix: $+ * + A B D / E + F * A D C$

Postfix: $A B + D * E F A D * + / + C +$

10) Discuss how DeQue(Double Ended Queue) is different from a linear queue

Feature	Double Ended Queue (Deque)	Linear Queue
Insertion Points	Can insert elements at both ends (front and rear).	Can insert elements only at the rear.
Deletion Points	Can delete elements from both ends (front and rear).	Can delete elements only from the front.
Flexibility	More flexible due to two-way insertion and deletion.	Less flexible, restricted to FIFO (First In, First Out).
Use Cases	Useful for applications like undo operations, and task scheduling where both ends are accessed.	Used in typical queue scenarios such as printer queues, where only one end (rear) is accessed for insertion.
Complexity	Generally more complex due to dual-end operations.	Simpler in implementation.
Examples	Deque operations in programming, job scheduling, undo functionality.	Linear queues for customer service, printer queues.

11) List out the applications of a) stack b) queues

a) Applications of Stack:

- **Expression Evaluation and Conversion** (Infix to Postfix/Prefix)
- **Backtracking Algorithms** (e.g., solving mazes, puzzles)
- **Function Call Management** (managing function calls in recursion)
- **Undo/Redo Operations** in text editors
- **Syntax Parsing** (in compilers for matching parentheses, braces)
- **Memory Management** (used in managing local variables in languages like C)
- **Browser History Navigation** (going back and forward in browser tabs)

b) Applications of Queue:

- **CPU Scheduling** (managing processes in operating systems)
- **Breadth-First Search (BFS)** in graphs and trees
- **Print Spooling** (managing printing jobs)
- **Customer Service Systems** (managing queues in banking, ticket booking)
- **Data Streaming** (for processing continuous data streams)
- **Packet Scheduling** in networking (managing data transmission)
- **Real-time Task Management** in operating systems (using priority queues)

Note : Write any 4 from each stack and queue applications

12) Discuss the use of bitfields with an example

Bitfields are used in programming to store data in a more memory-efficient way by allocating a specific number of bits to a variable within a structure.

This allows for more compact storage, especially when memory is constrained, and the values being stored do not need the full range of a standard data type.

Ex :-

```
struct {  
  
    unsigned int day : 5;  
  
    unsigned int month : 4;  
  
    unsigned int year : 7;  
  
} date;
```


13) Differentiate between arrays and Linked list.

Feature	Arrays	Linked Lists
Structure	Contiguous block of memory	Non-contiguous memory with pointers
Size	Fixed size (defined at declaration)	Dynamic size (can grow or shrink)
Memory Utilization	Memory allocated at compile time (static)	Memory allocated at runtime (dynamic)
Access Time	Fast access ($O(1)$) using index	Slow access ($O(n)$) as elements are traversed sequentially
Insertion/Deletion	Expensive; shifting required ($O(n)$)	Efficient ($O(1)$ for head, $O(n)$ for others)
Memory Overhead	No extra memory overhead	Extra memory for storing pointers

LAQ'S :

1) Explain Stack, its operations and write its implementation using arrays.

Stack:

A **stack** is a linear data structure that follows the **Last In, First Out (LIFO)** principle, meaning the element added last is the first one to be removed. The stack is often compared to a stack of plates, where you can only add or remove the top plate.

Basic Operations on a Stack:

3. **Push:** Adds an element to the top of the stack.
4. **Pop:** Removes and returns the top element from the stack.

5. **Peek/Top:** Returns the top element without removing it.
6. **isEmpty:** Checks if the stack is empty.
7. **isFull:** Checks if the stack is full (in case of fixed-size arrays).

Note: Code for Implementation of stack given in C.W

2) Describe Queue, and write the implementation of following operations on Queue

using arrays.

(i) enqueue (ii) dequeue (iii) display

Queue:

A **Queue** is a linear data structure that operates on the **First In, First Out (FIFO)** principle, meaning the element that is inserted first is the one to be removed first. A queue is similar to a line of people waiting at a ticket counter, where the person who comes first is served first.

Key Features of Queue:

- **FIFO Order:** The first element added to the queue will be the first to be removed.
- **Two Ends:** A queue has two ends:
 - **Front:** The end from which elements are removed (dequeue).
 - **Rear:** The end where elements are added (enqueue).

Basic Operations on a Queue:

- **Enqueue:** Adds an element to the rear of the queue.
- **Dequeue:** Removes and returns the element from the front of the queue.
- **Front:** Returns the element at the front without removing it.
- **isEmpty:** Checks if the queue is empty.
- **isFull:** Checks if the queue is full (in case of fixed-size queues).

Note: Code for Implementation of queue given in C.W

3) List out disadvantages of linear queue over Circular queue and Explain about Circular Queue.

Disadvantage	Description
Inefficient Memory Utilization	In a linear queue, even when elements are dequeued, the space at the front is wasted and cannot be reused, leading to inefficient memory utilization.
Queue Overflow	In a linear queue, even if there is space at the front (due to dequeued elements), the queue may still encounter overflow if the rear reaches the end, whereas a circular queue wraps around and utilizes free space.
Limited Capacity	A linear queue has a fixed size and does not efficiently reuse the space of dequeued elements, limiting the effective capacity even if some elements have been removed.
Shifting Elements	In certain implementations of linear queues, when an element is dequeued, all remaining elements must be shifted forward, resulting in additional time complexity ($O(n)$). This is avoided in circular queues.
Fixed Ends	In a linear queue, insertion can only happen at the rear and deletion only at the front, which restricts flexibility compared to the circular queue, where both ends are dynamically reused.

Circular queue :

A **Circular Queue** is a variation of a linear queue where the last position is connected back to the first position to form a circle.

This circular arrangement makes more efficient use of space, as elements can be added to the front once the rear reaches the end of the queue.

Basic Operations in Circular Queue:

- **Enqueue:** Adds an element to the rear of the queue, wrapping around to the front if the rear is at the last position and space is available.
- **Dequeue:** Removes an element from the front of the queue, wrapping around to the beginning if the front is at the last position.
- **Front:** Returns the element at the front of the queue without removing it.
- **isEmpty:** Checks if the queue is empty (i.e., front and rear pointers are at the same position).

- **isFull:** Checks if the queue is full (i.e., when the next position for the rear pointer is the front).

4) Explain Infix to Postfix Conversion using Stack. Apply the steps and convert the following expression: $A * B - (C + D) - E^F$ (Note: $\$$ = power)

Infix Expression: $A * B - (C + D) - E^F$

Infix to Postfix Conversion Using Stack:

Infix expression is the standard form of writing expressions, where operators are placed between operands (e.g., $A + B$).

Postfix expression (also known as Reverse Polish Notation - RPN) places operators after operands (e.g., $A \quad B \quad +$).

Explanation :

- **Initialize an empty stack** for operators and an empty list for the result (postfix expression).
- **Scan the infix expression from left to right.**
- **For each character** in the expression:
 - **Operand:** Directly append it to the result (postfix expression).
 - **Opening parenthesis (:** Push it onto the stack.
 - **Closing parenthesis):** Pop from the stack to the result until an opening parenthesis (is encountered.
 - **Operator:**
 - Pop operators from the stack to the result until the stack is empty or an operator of lower precedence is on top of the stack.
 - Then push the current operator to the stack.
- **After scanning** the infix expression, pop any remaining operators from the stack and append them to the result.
- **Operator precedence:**
 - $\wedge$ (Exponentiation) has the highest precedence.
 - $*$, $/$ have the next higher precedence.
 - $+$, $-$ have the lowest precedence.
- **Associativity:**
 - $\wedge$ is **right-associative** (evaluates from right to left).
 - $*$, $/$, $+$, $-$ are **left-associative** (evaluates from left to right).

Steps :

1	Read A		A
2	Read +	+	A
3	Read B	+	A B
4	Pop and append + (End of subexpression)		A B *
5	Read -	-	A B *
6	Read (	- (	A B *
7	Read C	- (	A B * C
8	Read +	- (+	A B * C
9	Read D	- (+	A B * C D
10	Pop and append + (End of subexpression)	- (	A B * C D +
11	Pop (	-	A B * C D +
12	Read -	- -	A B * C D +
13	Read E	- -	A B * C D + E
14	Read ^ (Exponentiation)	- - ^	A B * C D + E
15	Read F	- - ^	A B * C D + E F
16	Pop and append ^	- -	A B * C D + E F ^
17	Pop and append -	-	A B * C D + E F ^ -
18	Pop and append -		A B * C D + E F ^ - -

5) Illustrate the algorithm to evaluate a postfix expression. Evaluate the following expression using the algorithm:
5 6 2 + * 1 2 + + 4 1 - *

Algorithm to Evaluate a Postfix Expression Using a Stack:

To evaluate a postfix expression (also called Reverse Polish Notation), we use a **stack** to store operands. The algorithm processes the expression from left to right, performing the following steps:

Steps for Evaluating a Postfix Expression:

8. Initialize an empty stack.

9. **Scan the postfix expression** from left to right.
10. **For each element** in the expression:
 - **Operand:** If the element is an operand, push it onto the stack.
 - **Operator:** If the element is an operator, pop the required number of operands from the stack, apply the operator, and push the result back onto the stack.
11. **Continue this process** until the entire expression is processed.
12. **At the end**, the stack should contain one element, which is the result of the expression.

Operators:

- $+$, $-$, $*$, $/$, $^$ (addition, subtraction, multiplication, division, exponentiation).

Example: Evaluate the Postfix Expression 5 6 2 + * 1 2 + + 4 1 - *

Steps:

Step	Action	Stack
1	Read 5 (operand)	Push 5
2	Read 6 (operand)	Push 6
3	Read 2 (operand)	Push 2
4	Read + (operator)	Pop 6 and 2, push $6 + 2 = 8$
5	Read * (operator)	Pop 5 and 8, push $5 * 8 = 40$
6	Read 1 (operand)	Push 1
7	Read 2 (operand)	Push 2
8	Read + (operator)	Pop 1 and 2, push $1 + 2 = 3$
9	Read + (operator)	Pop 40 and 3, push $40 + 3 = 43$
10	Read 4 (operand)	Push 4
11	Read 1 (operand)	Push 1
12	Read - (operator)	Pop 4 and 1, push $4 - 1 = 3$
13	Read * (operator)	Pop 43 and 3, push $43 * 3 = 129$

Final Stack State:

The stack contains a single value: 129.

Result:

The result of evaluating the postfix expression 5 6 2 + * 1 2 + + 4 1 - * is **129**.

6) Illustrate Nested structures with appropriate examples

Nested Structures in C

Nested structures refer to a structure defined within another structure. This allows you to create complex data types that can hold related data together in an organized manner. Nested structures are useful for modeling real-world entities that have multiple attributes.

Ex:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
struct Name {
```

```
    char firstName[20];
```

```
    char lastName[20];
```

```
};
```

```
struct Address {
```

```
    char street[50];
```

```
    char city[20];
```

```
    char state[20];
```

```
    int zipCode;
```

```
};
```



```
struct Student {  
    struct Name name;    // Nested Name structure  
    struct Address address; // Nested Address structure  
    int studentID;  
};
```

7) Explain various methods to pass structure to function with appropriate examples

1. Passing Structure by Value

When a structure is passed by value, a copy of the structure is made, and the function works on this copy. Any changes made to the structure inside the function do not affect the original structure.

```
#include <stdio.h>
```

```
// Define a structure
```

```
struct Point { int x; int y; };
```

```
// Function to print the coordinates
```

```
void printPoint(struct Point p) {  
    printf("Point coordinates: (%d, %d)\n", p.x, p.y);  
}
```

```
int main() {
```

```
    struct Point pt = {10, 20};
```

```
    printPoint(pt); // Passing by value

    return 0;

}
```

2. Passing Structure by Reference

Passing a structure by reference means passing a pointer to the structure. This allows the function to access and modify the original structure.

```
#include <stdio.h>

struct Point {

    int x; int y; };

// Function to modify the coordinates

void modifyPoint(struct Point *p) {

    p->x += 5; // Modify the original structure

    p->y += 10;

}

int main() {

    struct Point pt = {10, 20};

    modifyPoint(&pt); // Passing by reference (using address)

    printf("Modified Point coordinates: (%d, %d)\n", pt.x, pt.y);

    return 0; }
```

8) Write an algorithm to perform operations on DEQUE such as insert, delete, and display.

Note : Answer given in cw

9) Explain array of structures can be accessed using pointers with example.

```
#include <stdio.h>

#include <string.h>

#define MAX_STUDENTS 5

// Define a structure for Student

struct Student {

    char name[30];

    int rollNo;

};

// Function to display student details

void displayStudents(struct Student* ptr, int count) {

    for (int i = 0; i < count; i++) {

        printf("Student %d:\n", i + 1);

        printf("Name: %s\n", (ptr + i)->name);

        printf("Roll No: %d\n", (ptr + i)->rollNo);

        printf("\n");

    }

}
```

```

int main() {

    // Declare an array of structures

    struct Student students[MAX_STUDENTS];

    for (int i = 0; i < MAX_STUDENTS; i++) {

        printf("Enter name for student %d: ", i + 1);

        fgets(students[i].name, sizeof(students[i].name), stdin);

        students[i].name[strcspn(students[i].name, "\n")] = 0; // Remove newline

        printf("Enter roll number for student %d: ", i + 1);

        scanf("%d", &students[i].rollNo);

        getchar(); // To consume the newline character left by scanf

    }

    printf("\nStudent Details:\n"); displayStudents(students, MAX_STUDENTS);return 0;}

```

Explanation:

- **Structure Definition:** The Student structure holds a name and roll number.
- **Array Declaration:** We declare an array of Student structures called students.
- **Input Loop:** We loop through the array, allowing the user to input student names and roll numbers.
- **Pointer Access:**
 - In the displayStudents function, we use a pointer to the Student structure array.
 - The pointer arithmetic (ptr + i) accesses each structure, and the -> operator is used to access members of the structure.
- **Display:** Finally, we print the details of each student.

10) Write detailed implementation of all operations on Singly Linked List (chain).

To be continued.....